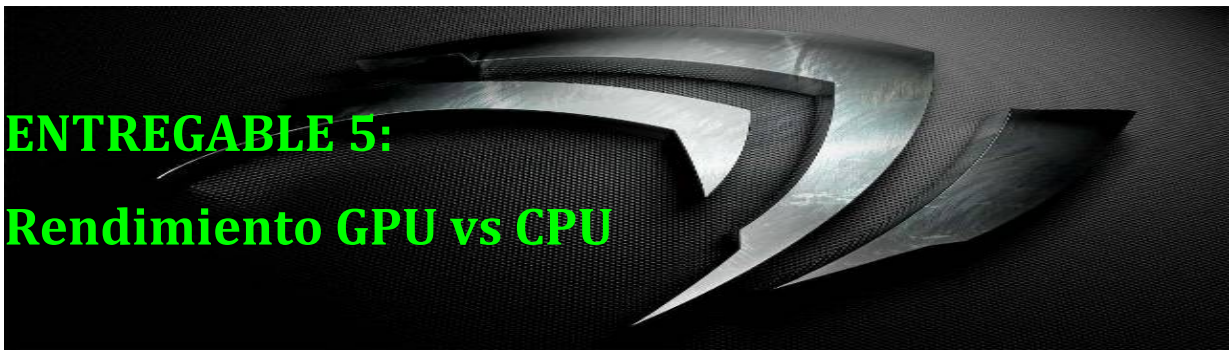


1. Objetivo

Calcular la ganancia en rendimiento de la GPU respecto de la CPU.

2. Ejercicio

Ordenar de menor a mayor un vector de *N* elementos aleatorios comprendidos entre 1 y 30 utilizando el método de ordenación por rango.

El programa debe medir el tiempo empleado en la ordenación tanto en la CPU como en la GPU para los diferentes valores de *N* y calcular la ganancia en rendimiento de la GPU respecto de la CPU (ver tabla).

El programa deberá mostrar por pantalla las principales características del *device*, el tamaño del vector que se va a ordenar, el número de hilos por bloque y el número de bloques lanzados así como los tiempos empleados en la ordenación.

El programa podrá bien pedir al usuario el tamaño *N* del vector que se desea ordenar (figura 5.1) o bien hacer todos los cálculos en una sola ejecución (Figura 5.2).

En este ejercicio se debe entregar la tabla con los tiempos, una gráfica representando los tiempos de ejecución en función de *N* y otra gráfica representando la ganancia de la GPU respecto de la CPU para los diferentes valores de *N*.

TABLA 4. Comparativa de rendimiento GPU/CPU.

N	32	64	128	256	512	1024	2048	4096
Tiempo GPU (ms)								
Tiempo CPU (ms)								
$G_{GPU/CPU}$								

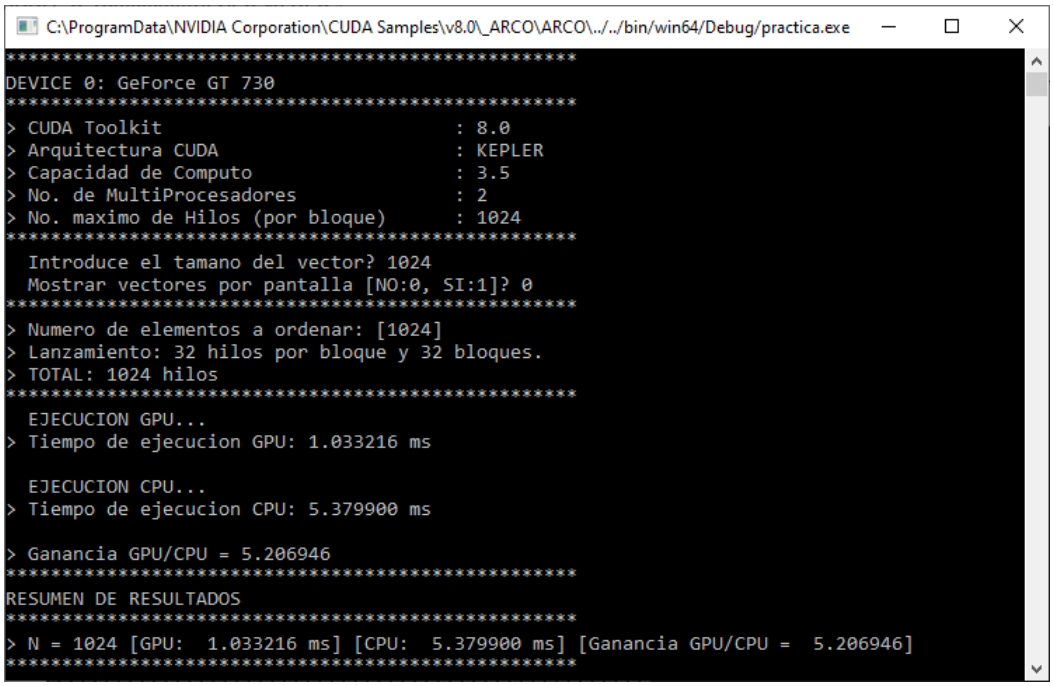


Figura 5.1.: Salida por pantalla de una posible solución.

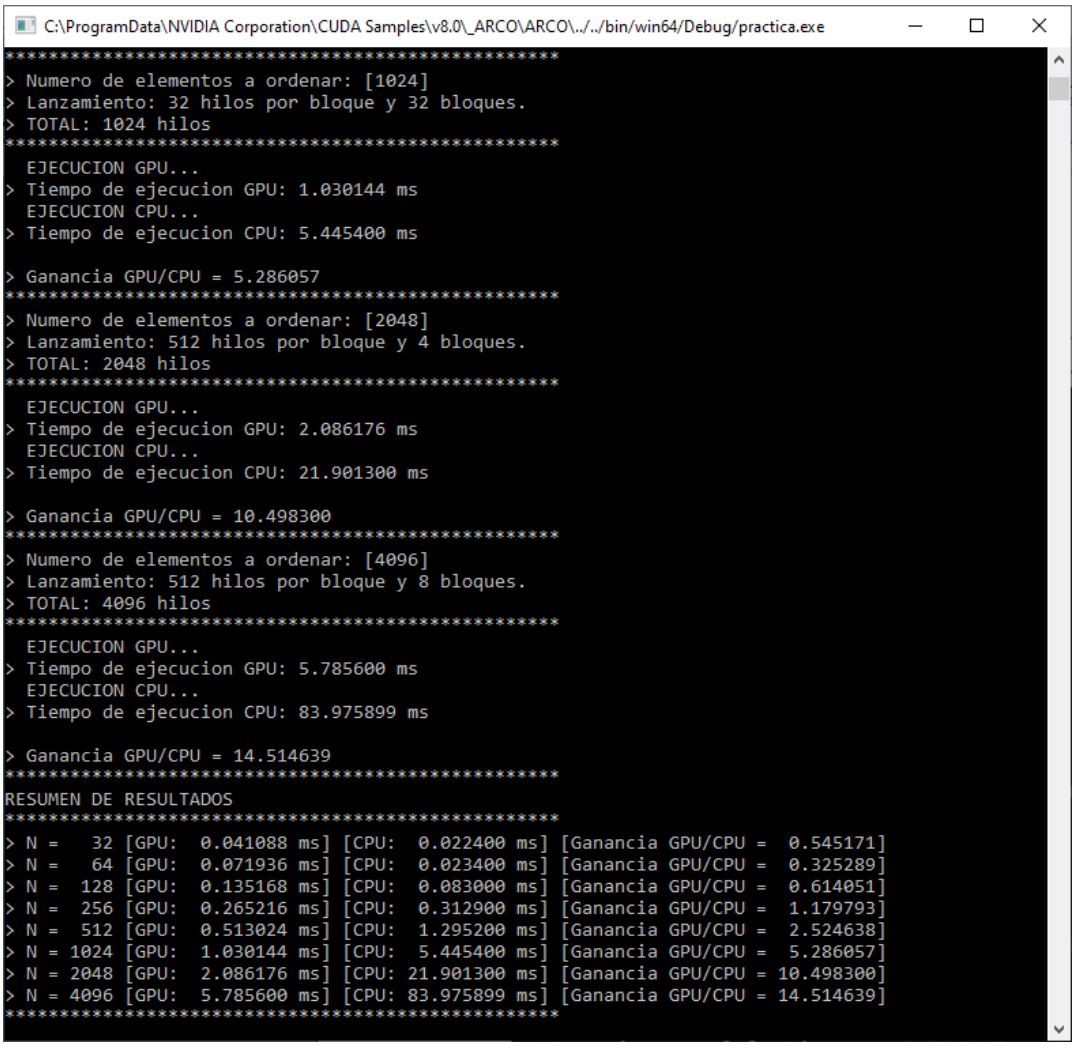
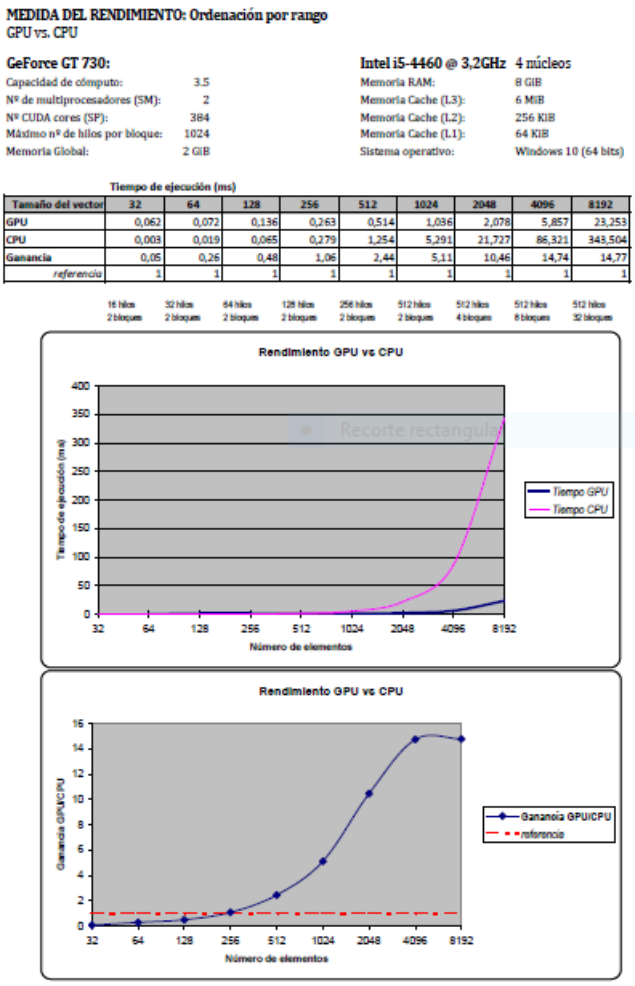


Figura 5.2.: Fragmento de la salida por pantalla de otra posible solución.

Los tiempos de ejecución en la GPU dependen tanto de la capacidad de cómputo de la GPU como del aprovechamiento de sus recursos. Esto quiere decir que utilizando un mismo *device* se pueden obtener tiempos muy distintos dependiendo de la configuración del *kernel*, esto es, dependiendo del número de bloques lanzados y del tamaño de los mismos.

Por otro lado, los tiempos de ejecución en la CPU también dependen del sistema anfitrión (*host*), lo que implica que la ganancia final en rendimiento de la GPU respecto de la CPU va a estar condicionada por las características de la CPU (Figura 5.3).



MEDIDA DEL RENDIMIENTO: Ordenación por rango
GPU vs. CPU

GeForce GT 730:

Capacidad de cómputo:	3.5
Nº de multiprocesadores (SM):	2
Nº CUDA cores (SP):	384
Máximo nº de hilos por bloque:	1024
Memoria Global:	2 GB

Intel i5-4460 @ 3,2GHz 4 núcleos

Memoria RAM:	8 GB
Memoria Cache (L3):	6 MB
Memoria Cache (L2):	256 KB
Memoria Cache (L1):	64 KB
Sistema operativo:	Windows 10 (64 bits)

Tiempo de ejecución (ms)

Tamaño del vector	32	64	128	256	512	1024	2048	4096	8192
GPU	0,062	0,072	0,136	0,263	0,514	1,036	2,078	5,857	23,253
CPU	0,003	0,019	0,065	0,279	1,254	5,291	21,727	86,321	343,504
Ganancia	0,05	0,26	0,48	1,06	2,44	5,11	10,46	14,74	14,77
referencia	1	1	1	1	1	1	1	1	1

Figura 5.3.: Comparativa en rendimiento entre una GPU y una CPU.

3. Temporización CPU

Como hemos mencionado anteriormente, si queremos medir el tiempo de ejecución de un programa ejecutado en la CPU debemos recurrir a funciones de la biblioteca de C. Sin embargo, para conseguir una mayor resolución temporal vamos a ver en el siguiente ejemplo cómo utilizar eventos del sistema operativo para medir el tiempo de cálculo de una multiplicación entre dos matrices cuadradas:

```
////////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <time.h>

#ifdef __linux__
#include <sys/time.h>
typedef struct timeval event;
#else
#include <windows.h>
typedef LARGE_INTEGER event;
#endif

////////////////////////////////////////////////////////////////////////////
// declaracion de funciones
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void setEvent(event *ev)
/* Descripcion: Genera un evento de tiempo */
{
#ifdef __linux__
  gettimeofday(ev, NULL);
#else
  QueryPerformanceCounter(ev);
#endif
}

__host__ double eventDiff(event *first, event *last)
/* Descripcion: Devuelve la diferencia de tiempo (en ms) entre dos eventos */
{
  #ifdef __linux__
  return
  ((double)(last->tv_sec + (double)last->tv_usec/1000000)-
  (double)(first->tv_sec + (double)first->tv_usec/1000000))*1000.0;
  #else
  event freq;
  QueryPerformanceFrequency(&freq);
  return ((double)(last->QuadPart - first->QuadPart) / (double)freq.QuadPart) * 1000.0;
  #endif
}

////////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
// declaraciones
float *hst_A, *hst_B, *hst_C;
const int TAM = 1000; // Tamano de las matrices: TAMxTAM
// reserva en el host
hst_A = (float*)malloc(TAM*TAM*sizeof(float));
hst_B = (float*)malloc(TAM*TAM*sizeof(float));
hst_C = (float*)malloc(TAM*TAM*sizeof(float));

// inicializacion
srand((int)time(NULL));
for (int i = 0; i<TAM*TAM; i++)
{
  hst_A[i] = (float) rand() / RAND_MAX;
  hst_B[i] = (float) rand() / RAND_MAX;
  hst_C[i] = 0;
}
```

```
// TEMPORIZACION
// Mediante eventos del sistema. Precision: fracciones de 1 ms.
// La funcion setEvent() devuelve un evento de tiempo.
// La funcion eventDiff() calcula la diferencia de tiempo (en milisegundos) entre dos
eventos.

event start; // variable para almacenar el evento de tiempo inicial.
event stop;  // variable para almacenar el evento de tiempo final.

// Multiplicacion de matrices
printf("*****************************************************\n");
printf("TEMPORIZACION CPU\n");
printf("*****************************************************\n");
printf("Multiplicando matrices %dx%d...", TAM,TAM);

// Iniciamos el contador
setEvent(&start); // marca de evento inicial

for (int i = 0; i<TAM; i++)
{
  for (int j = 0; j<TAM; j++)
  {
          hst_C[j + i*TAM] = 0;
          for (int k = 0; k<TAM; k++)
          {
                  hst_C[j + i*TAM] += hst_A[k + i*TAM] * hst_B[j + k*TAM];
          }
  }
}

// Paramos el contador
setEvent(&stop);// marca de evento final

printf("listo!\n");

// Intervalos de tiempo
double t_ms = eventDiff(&start, &stop); // diferencia de tiempo en ms
// Pasamos las unidades a segundos
double tiempo = t_ms/1000;              // tiempo en s
printf("Tiempo de ejecucion: %.8f s\n", tiempo);

// salida
printf("\npulsa INTRO para finalizar...");
getchar();
return 0;
}
```



Figura 5.4.: Salida por pantalla de la ejecución del código de ejemplo anterior.